



Skeletal Mesh.

+ Ask “What is skeletal mesh?”

Pro ▾





Skeletal mesh에 대해 알려줘. 우선 목차부터 보여줘.

💡 5s 동안 생각함

네, skeletal mesh에 대해 설명하기 전에 학습 목차부터 보여드리겠습니다.

Skeletal mesh 학습 목차:

1. USkeletalMeshComponent, USkinnedMeshComponent
2. Skinning의 수학적 원리
3. FBX импорт 파이프라인 (FBX SDK, FbxLoader)
4. Skeletal Mesh Viewer



+ Ask “What is skeletal mesh?”

Fast ▾





USkeletalMeshComponent, USkinnedMeshComponent에 대해 알려줘.

💡 5s 동안 생각함

USkinnedMeshComponent는 Bone에 의해 변형될 수 있는 Mesh Component 기반 클래스입니다. USkeletalMeshComponent는 이 클래스를 상속받아, 실제 Skeletal Mesh Animation을 재생하고 관리하는 Component입니다.

정리하면 다음과 같습니다.



USkinnedMeshComponent: Bone, Socket, Skin Weight, Skeletal Mesh 렌더링에 필요한 기본 기능을 제공합니다.

USkeletalMeshComponent: Animation Blueprint, Animation Sequence, Pose 계산, Montage 재생 등 실제 캐릭터 애니메이션 시스템을 담당하는 Component입니다.



+ Ask "What is skeletal mesh?"

Fast ▾

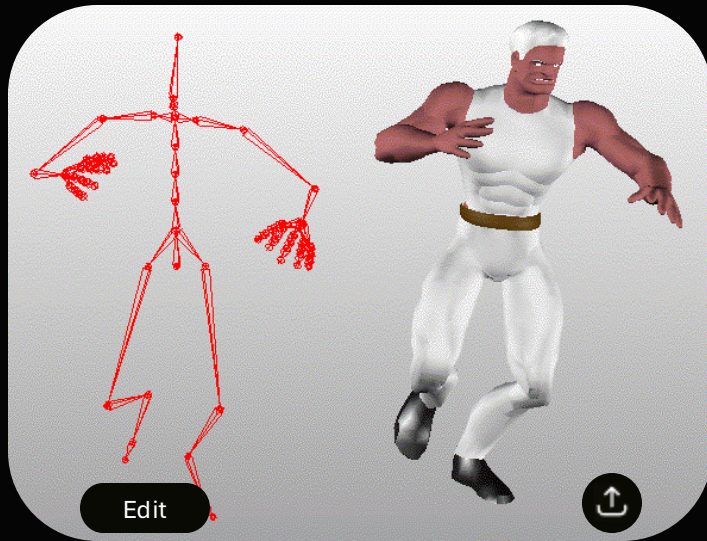




Skinning이 뭐야?

💡 3s 동안 생각함

Skinning은 Vertex를 직접 애니메이션 하는 방식이 아니라, Vertex를 Bone이라는 좌표계에 연결하고, Bone의 움직임을 따라가게 만드는 시스템입니다.



+ Ask "What is skeletal mesh?"

Fast ▾





Skinning의 수학적 원리에 대해 알려줘.

💡 31s 동안 생각함

Bone이 정점이 아니라 좌표계라는 점이 중요합니다.

Skeletal Mesh에는 두 종류의 데이터가 존재합니다:

Mesh Vertex: Mesh Space 기준으로 저장



Bone Transform: Parent Bone 기준 Local Transform으로 저장

Vertex가 Mesh Space에 있기 때문에 Vertex를 Bone과 바인딩해 주기 위해서 우선 Vertex를 Bone Space 기준으로 변환해야 합니다.

Mesh(bind) → Bone(bind)



+ Ask “What is skeletal mesh?”

Pro ▾





Bind Pose 시점에서 Bone은 Mesh 기준 위치와 회전 정보를 가지고 있습니다.

`BindPoseMatrix` = Bone → Mesh 변환



그런데 우리가 필요한 건:

Mesh → Bone 변환



그래서 `BindPoseMatrix`의 역행렬을 구해줍니다:

`</>` C++



```
InverseBindPose = BindPoseMatrix.GetInverse();
```

이러면 모든 Vertex들을 각 Bone Space의 좌표로 치환할 수 있습니다.

+ Ask “What is skeletal mesh?”

Pro ▾

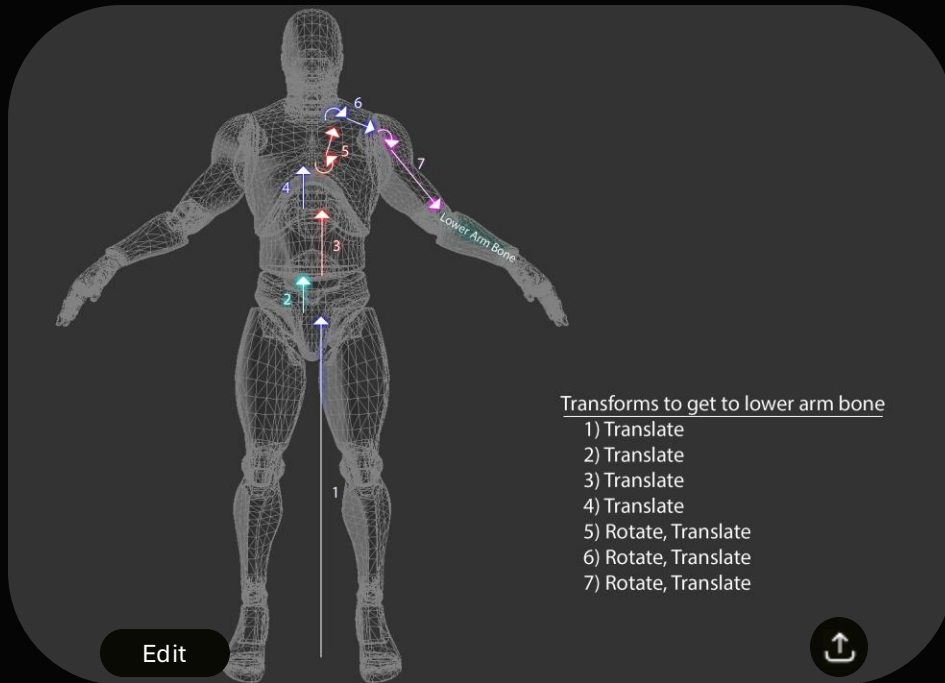




이제 우리가 필요한 건 각 Bone Space에 존재하는 Vertex들에 Current Pose를 적용해서 다시 Mesh Space로 보내는 겁니다. 이를 위해서는 CurrentBoneTransform이 필요합니다.

Animation은 Bone의 Local Motion만 저장하고, Runtime에서는 이를 계층적으로 누적해 CurrentBoneTransform을 계산합니다.

$\text{ChildGlobal} = \text{ChildLocal} * \text{ParentGlobal}$



+ Ask "What is skeletal mesh?"

Pro ▾





이렇게 계산된 `CurrentBoneTransform`(Bone Global Transform)은 현재 프레임에서 Bone이 Mesh 기준 어디에 있는지를 나타냅니다.

`</>` C++



```
// 1. Skinning Matrix 계산 (Bind Pose 정점을 현재 애니메이션 포즈로 변환)
SkinningMatrix = InverseBindPose[BoneIndex] * CurrentBoneTransform[BoneIndex];

// 2. CPU Skinning (영향을 주는 모든 본의 변환 결과에 가중치를 곱해 합산)
SkinnedPosition += SkinningMatrix.TransformPosition(Vertex.Position) * Weight;
```



+ Ask "What is skeletal mesh?"

Pro



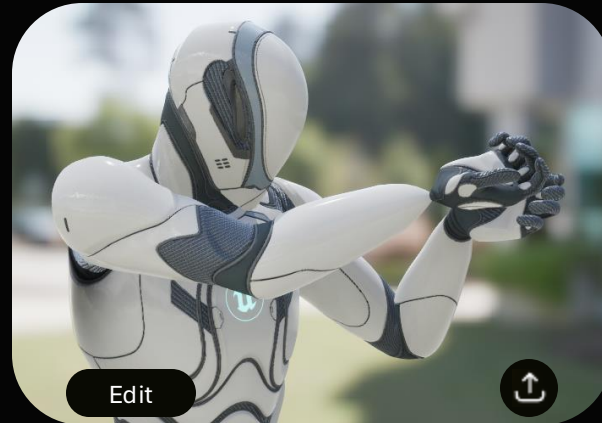
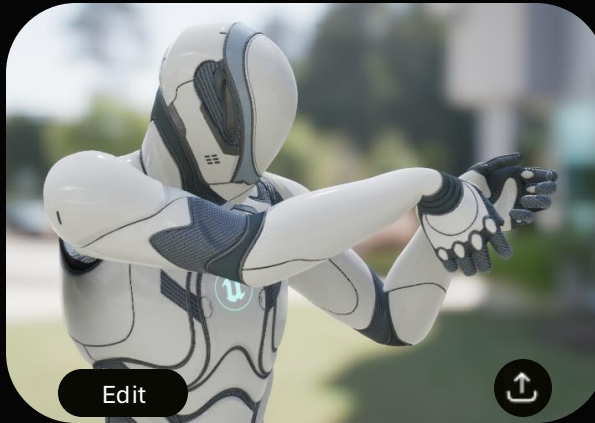


근데 LBS를 적용할 때 SkeletalMesh의 모양이 이상할 때가 있던
데?

💡 17s 동안 생각함

맞습니다. 이런 **LBS (Linear Blend Skinning)**은 두 가지 문제점이 있습니다.
아래 이미지처럼 손목을 굽히거나 돌릴 때 문제가 발생합니다.

(왼쪽) Joint Collapse : 정점들이 중간 위치로 끌려가면서 관절 부위가 납작해짐
(오른쪽) Candy Wrapper : 메시의 정점들이 회전할 때 선형 보간되어 사탕 포장지처럼 꼬임



+ Ask "What is skeletal mesh?"

Pro ▾





흠, 근데 이걸 어떻게 엔진에 가져오지? 찾아보니까 FBX가 자주 사용된다
는데.

💡 5s 동안 생각함

아하, FBX Import에 대한 질문이군요!

단순히 Mesh 정보만 있던 OBJ와 달리 FBX는 아래와 같은 정보도 같이 들어 있습니다.

따라서 FBX Import는 단순히 모델을 엔진에 올리는 것에 그치지 않고 사용 가능한 구조로
변환해야 합니다.

Mesh
Skeleton
Bone Hierarchy
Skinning Weight
Material
Animation



+ Ask “What is skeletal mesh?”

Fast ▾





그걸 어떻게 하는데?

💡 21s 동안 생각함

FBX는 다음과 같이 구성됩니다:

```
FbxScene
├── RootNode
│   ├── FbxNode Tree
│   │   ├── Transform
│   │   ├── NodeAttribute
│   │   │   ├── FbxMesh
│   │   │   ├── FbxSkeleton
│   │   │   ├── FbxCamera
│   │   │   └── FbxLight
│   │   ├── Material References
│   │   └── Children
│   ├── Pose List
│   │   └── FbxPose
│   │       └── Node → Bind Matrix
│   ├── Material / Texture Objects
│   └── Animation Stacks
```



+ Ask “What is skeletal mesh?”

Pro ▾

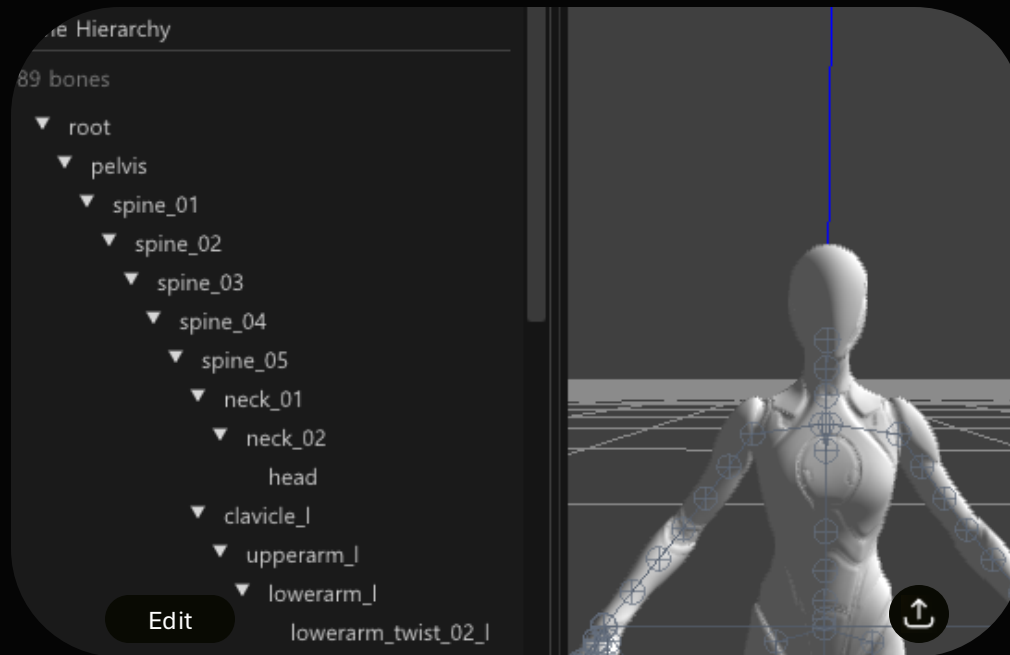




FbxScene 의 노드 트리를 순회하며 아래와 같은 Import 타입을 분류합니다.

여기서 FBX의 NodeAttribute 를 보고 판단합니다.

만일 eSkeleton 이라면 Bone Node 이므로 아래처럼 Bone 계층 구조를 만듭니다.



+ Ask "What is skeletal mesh?"

Pro ▾

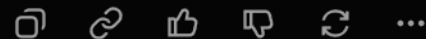
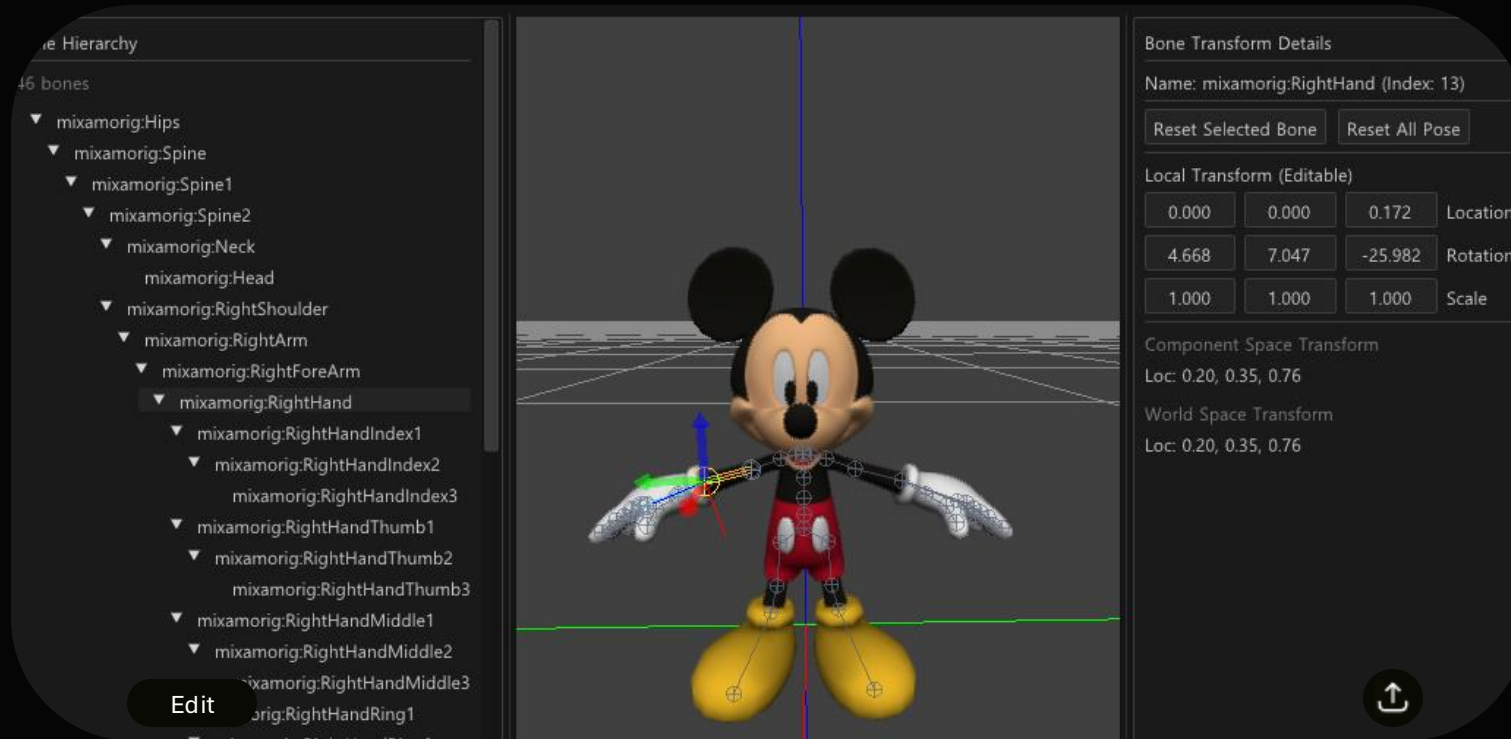




SkeletalMesh Viewer는 어떤 식으로 구성할지 이미지로 보여
줘.

💡 20s 동안 생각함

이런 식으로 구성해 보는 것은 어떨까요?



+ Ask "What is skeletal mesh?"

Pro

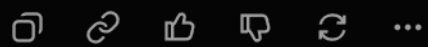




우리 시연할 거니까 잘 봐.

💡 1s 동안 생각함

네, 화이팅하세요.



+ 너 T야?

Pro ▾

